

به نام خدا

پیش گزارش آزمایش 9 –  
پیاادهسازی TCAM – درس  
آزمایشگاه طراحی سیستم  
های دیجیتال

امیر محمد شربتی (402106112) – پوریا رحمانی (402111418)

1404/10/14

در این آزمایش باید تلاش کنیم تا یک TCAM (Ternary Content Addressable Memory) یا همان حافظه‌ی شرکت‌پذیر نوع سه گانه را پیاده سازی کنیم. در ابتدا توضیح مختصری در مورد این نوع حافظه می‌دهم، سپس در مورد پیاده سازی ماژول ها، در مورد چالش ها و در مورد صحت پیاده سازی و سنتز پذیری مدار توضیحاتی ارائه می‌دهم:

## مقدمه

حافظه‌های Content Addressable Memory (CAM) نوعی از حافظه‌ها هستند که برخلاف حافظه‌های معمولی (RAM) به جای دسترسی با آدرس، داده‌ها را با مقدار جستجو می‌کنند.

در CAM معمولی (Binary CAM) هر بیت حافظه می‌تواند 0 یا 1 باشد و ورودی جستجو با تمام سطرهای حافظه مقایسه می‌شود. Ternary CAM (TCAM) نسخه پیشرفته‌تری است که هر بیت می‌تواند سه حالت داشته باشد:  $0 - 1 - X$  ویژگی  $X$  (don't care) باعث می‌شود که بیت مورد نظر در مقایسه نادیده گرفته شود و فقط بیت‌های 0 یا 1 بررسی شوند. این قابلیت TCAM را برای کاربردهایی مانند سیستم‌های مسیریابی، فشرده‌سازی، بانک‌های داده و سیستم‌های هوشمند بسیار مناسب می‌کند.

## شرح آزمایش

هدف آزمایش پیاده‌سازی یک TCAM با 16 ثبات 16 بیتی است. پیاده سازی TCAM ما قابلیت‌های زیر را دارد:

- ذخیره و مدیریت مقادیر (value) و ماسک‌ها (mask) برای هر ثبات
- جستجوی سریع یک داده 16 بیتی
- حساب کردن خروجی match برای هر دیتا با تمام اعداد ذخیره شده در حافظه

همچنین پیاده سازی ما قابل سنتز است.

## ماژول نهایی TCAM

ماژول به صورت پارامتری طراحی شد تا اندازه هر ثبات (WIDTH) و تعداد ثبات‌ها (DEPTH) قابل تغییر باشد. همچنین یک سیگنال reset به صورت سنکرون اضافه شد تا حافظه در زمان شروع یا نیاز به پاک کردن حافظه، مقدار صفر دریافت کند.

## چگونگی پیاده سازی قابلیت Ternary

دقت شود برای اینکه قابلیت X یا همان don't care قابل پیاده سازی باشد، لازم است برای هر بیت، دو بیت معادل در نظر بگیریم. به این معنا که اگر آن بیت مشخص بود، بیت متناظر value صفر یا یک باشد و اگر X بود و اهمیتی نداشت، لازم است یک mask در نظر بگیریم که یک بودن آن متناظر با X بودن بیت عدد است و دیگر مقدار value اهمیتی ندارد.

به همین دلیل در پیاده سازی یک vector به نام value و دیگری به نام mask داریم. برای مثال مقداری که برای X01X ذخیره میکنیم، به این صورت است که  $mask[3] = mask[0] = 1$  و  $mask[2] = mask[1] = 0$  و همچنین در جاهایی که mask صفر است، باید مقدار value به درستی متناظر با بیت عدد باشد، یعنی  $value[2] = 1$  ,  $value[1] = 0$

برای اینکه بتوانیم در آزمایشگاه این مدار را بررسی کنیم، لازم است تا اعدادی را در حافظه ذخیره کنیم. بنابراین لازم است به عنوان ورودی آدرس و مقادیر ورودی را به مدار بدهیم تا حافظه را به صورت دستی پر کنیم. اگر صلاح باشد می توان این کار را به صورت hard code شده در قسمت reset کد انجام داد.

## منطق پیاده سازی

سیگنال های مهم مدار:

- value\_mem: آرایه ای از مقادیر داده TCAM  $\leq$  آرایه ای از vector ها
- mask\_mem: آرایه ای از ماسک ها که بیت های X را مشخص می کند.
- match: خروجی 16 بیتی که مشخص می کند کدام ثبات ها با داده ورودی تطابق دارد. مثلاً اگر بیت دوم و پنجم match یک باشند و مابقی صفر باشند، در این صورت یعنی مقدار ثبات دوم و پنجم با عدد ورودی مطابق هستند.
- search\_data: داده ورودی که باید تطابق آن با اعداد ثبات ها را بررسی کنیم.
- write\_enable, write\_addr, write\_value, write\_mask: اگر  $write\_enable = 1$  باشد، در این صورت در آدرس مشخص شده (در ثبات مشخص شده)، مقدار ورودی ( $write\_value$  ,  $write\_mask$ ) نوشته می شود.

منطق اصلی پیاده سازی، این بخش کد است:

```
// TCAM compare logic
always @(*) begin
    for (i = 0; i < DEPTH; i = i + 1) begin
        match[i] = & (mask_mem[i] | ~(search_data ^ value_mem[i]));
    end
end
```

منطق مقایسه به صورت ترکیبی است. هر داده ورودی، با تمام داده های ذخیره شده در حافظه، یعنی با تمام 16 ثبات مقایسه می شود. دلیل for همین است. اما مهمترین نکته تک خط درون حلقه است. به زبان ساده این خط می گوید که  $match[i]$  زمانی یک می شود (یعنی زمانی تطابق با ثبات  $i$  را گزارش می دهد) که یا بیت متناظر در ثبات  $X$  باشد ( $mask\_mem[i]=1$ ) یا اگر نباشد باید بیت متناظر عدد و ثبات برابر باشند ( $search\_data \wedge value\_mem[i] = 0$ ). اگر برای تمام بیت های دو عدد این حاصل یک باشد (که این به راحتی با یک  $\&$  پشت این عبارت پیاده سازی شده است که reduction operator نامیده می شود)، در این صورت تطابق با این ثبات گزارش می شود.

## ماژول تست (tcam\_tb)

تلاش شد تا یک testbench جامع و کامل بنویسیم که پوشش دهنده حالات بسیار باشد. ساختار ماژول بدین صورت است:

- تولید clock
- task ها (write\_entry و check\_search) برای نوشتن داده ها و بررسی خروجی
- تابع compute\_match برای محاسبه خروجی صحیح
- تست های دستی و تست های تصادفی (random tests)

روند تست تصادفی به این صورت است که برای هر مرحله 16 سطر TCAM با داده و ماسک تصادفی پر می شود. سپس ک داده جستجوی تصادفی تولید می شود. در نهایت خروجی ماژول با مقدار صحیح مقایسه می شود.

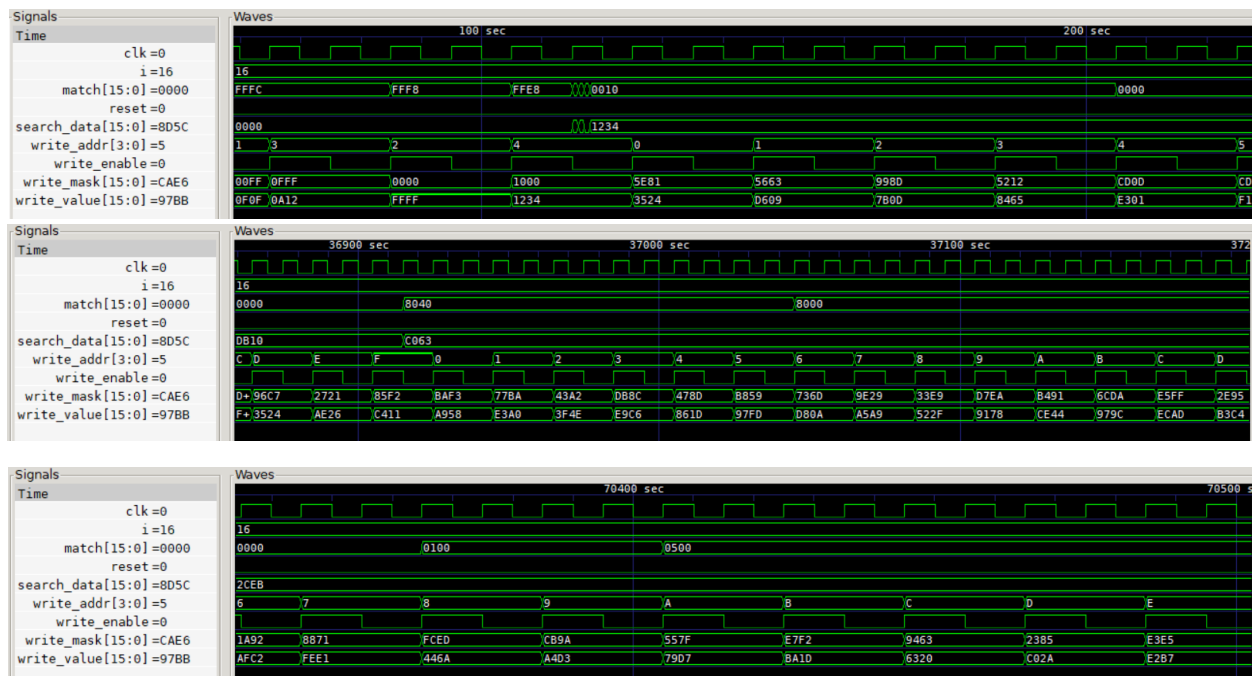
## نتایج

این هم نتایج تست است که مطابق انتظار است:

```
PS C:\Users\ASUS\OneDrive\Desktop\university\5\DSB_Lab\9\circuit> vvp tcam.vvp
VCD info: dumpfile tcam.vcd opened for output.
===== TCAM TEST START =====
Running directed tests...
Running random tests...
===== ALL TCAM TESTS PASSED =====
```

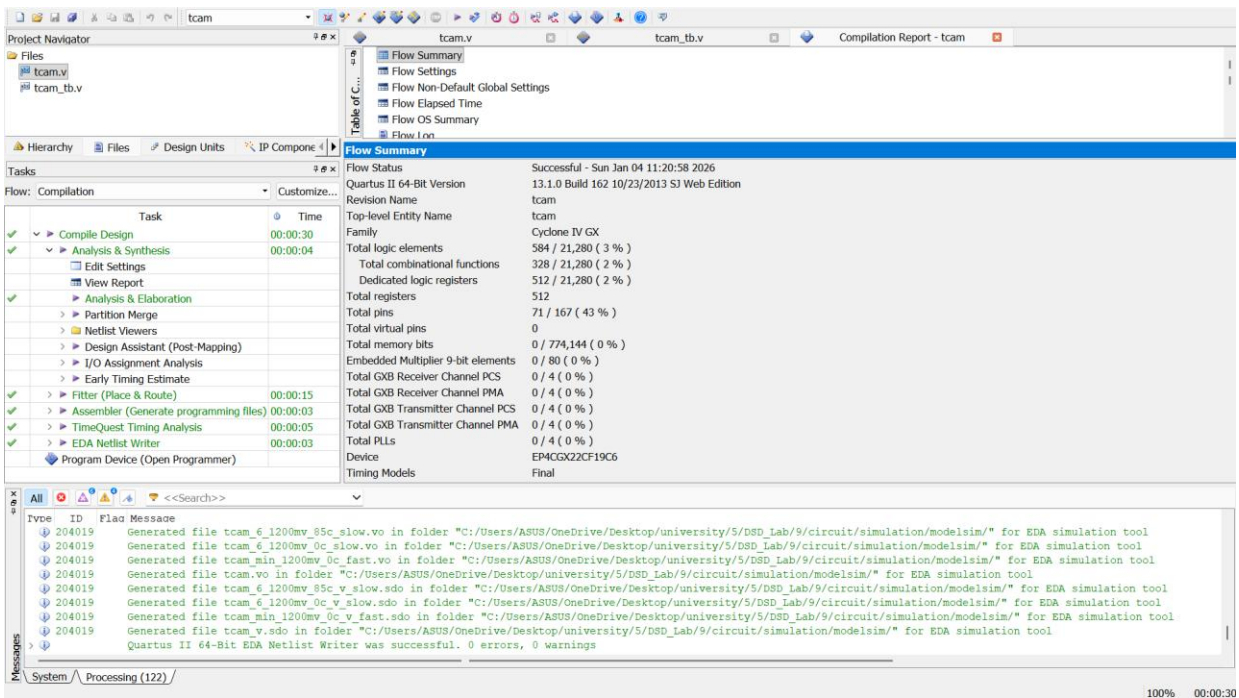
به دلیل اینکه تعداد بسیار زیادی داده تست شدند، امکان عکس گرفتن از کل waveform وجود نداشت. بنابراین فقط سه عکس به صورت تصادفی از بخش های مختلف آن را قرار می دهیم.

عکس اول مربوط به تست های دستی و عکس های دوم و سوم مربوط به تست های تصادفی است:

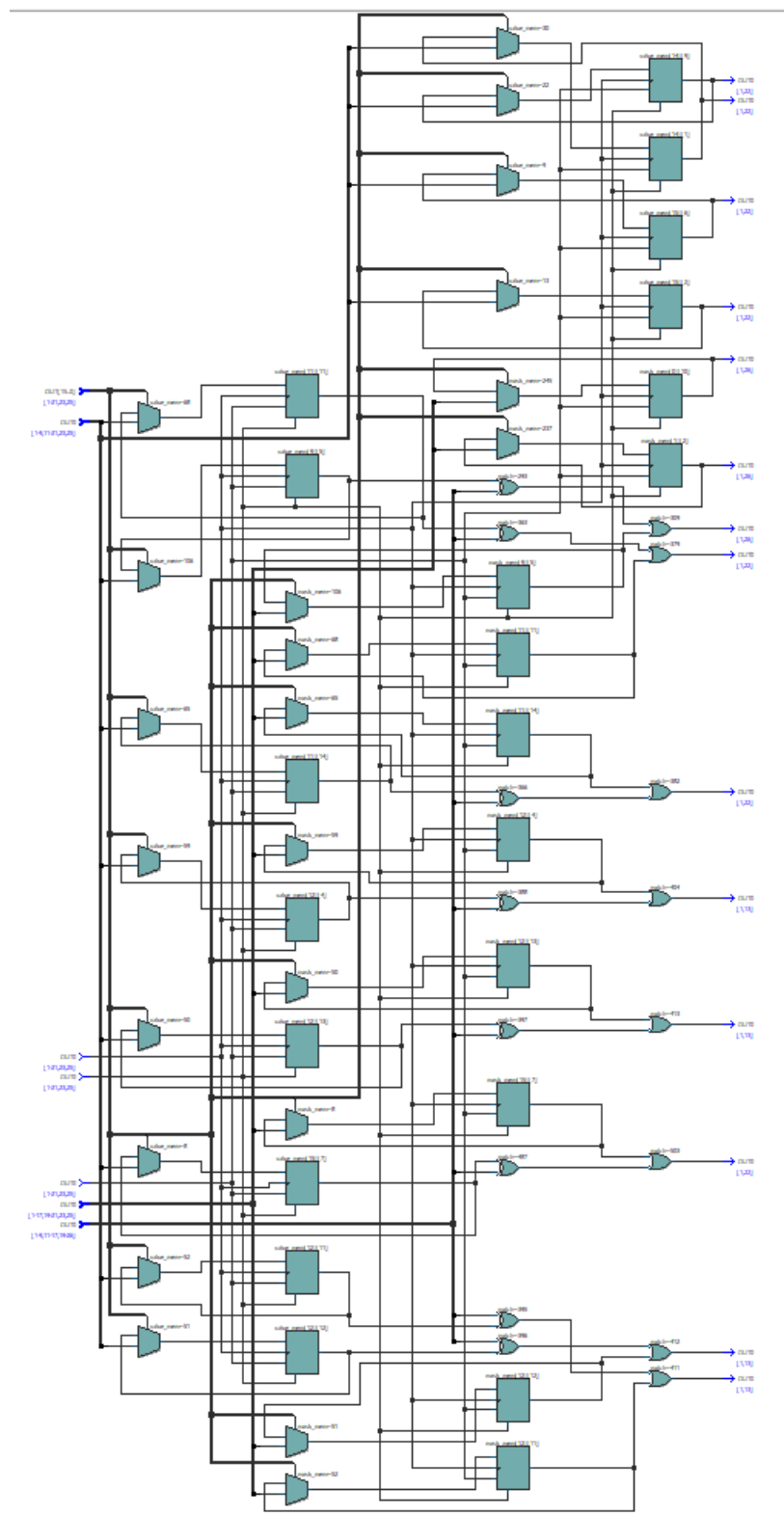


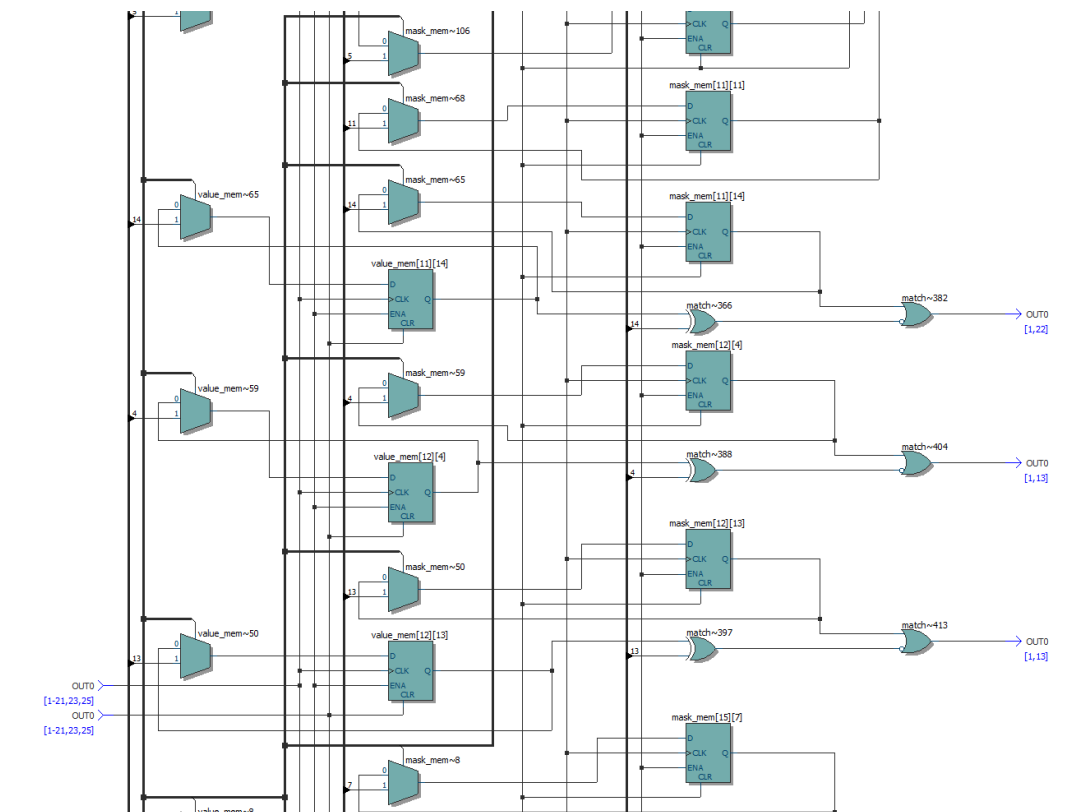
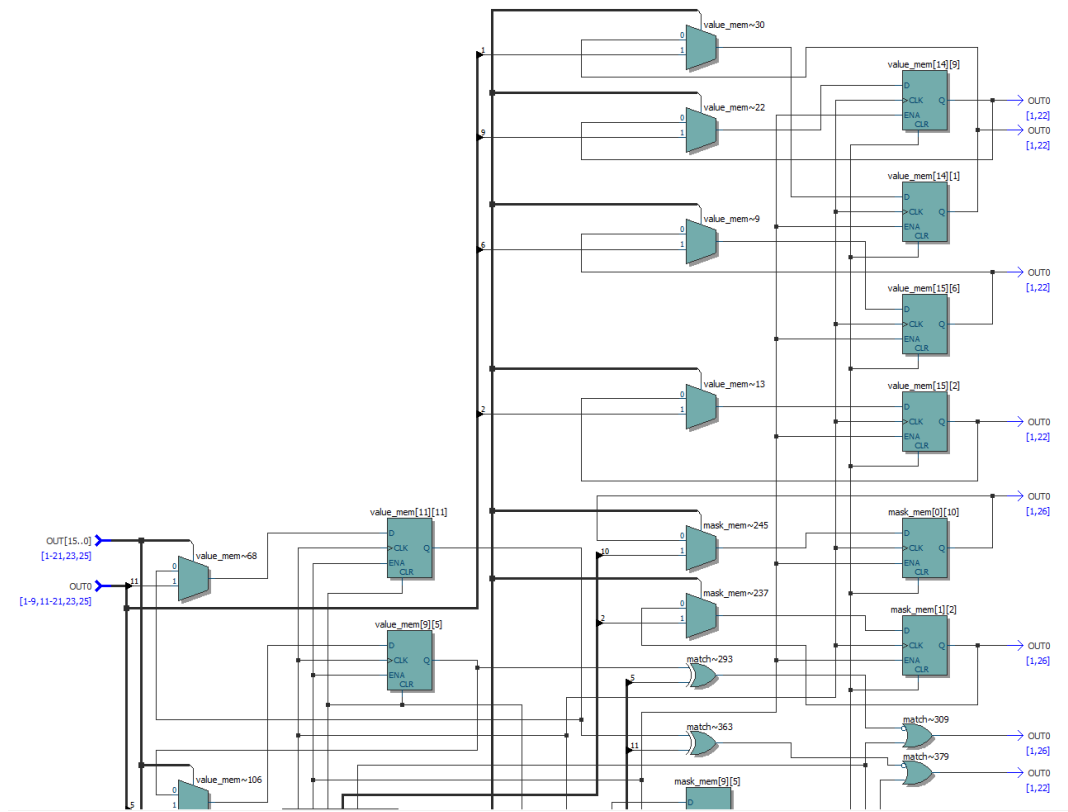
## سنتز پذیری

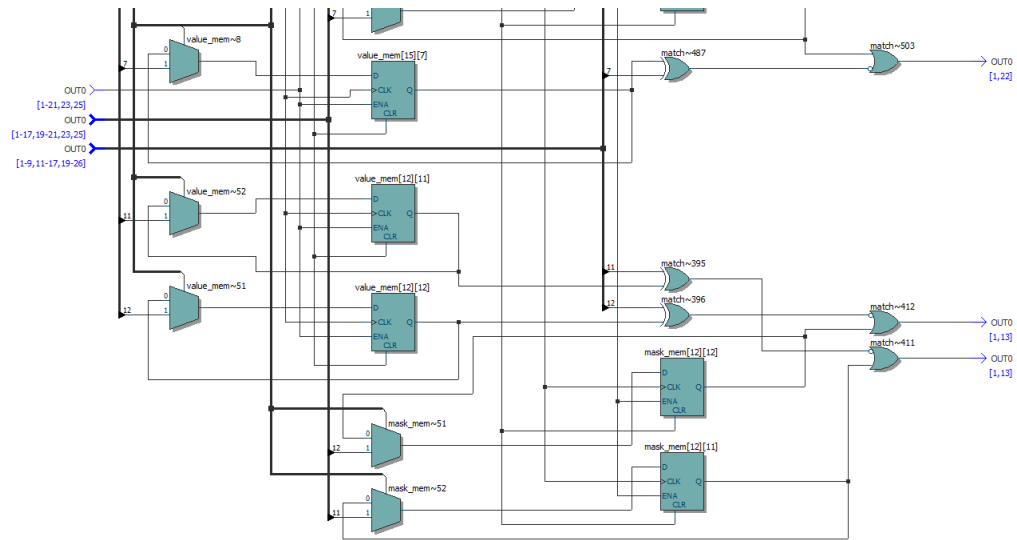
خدا رو شکر این پیاده سازی قابل سنتز است. در کوارتوس این مدار را اجرا کردیم. عکس زیر مربوط به data flow summary است:



در زیر عکس های RTL Viewer قرار دارد. عکس اول یک عکس کلی است ولی چون به نظر ریز است و جزئیات آن چندان معلوم نیست، بنده کمی بزرگ کردم و در سه عکس تکه تکه، در ادامه ی گزارش قرار دادم.







خدا رو شکر همه چیز مطابق انتظار است. تست ها نشان دهنده درستی مدار بودند. در نهایت هم دیدیم که کد قابل سنتز بود.

سایر فایل ها شامل کد های ورپلاگ و فایل های RTL viewer و Flow summary هم به پیوست ارسال شدند.